Apuntes · Examen Java · ERP Frescura Total

Eidan Carreño · Hecho a partir de tus errores reales en la práctica. Léelo de arriba abajo antes de empezar.

1 · Tus errores frecuentes (revísalos SIEMPRE antes de entregar)

NOMBRES **Mayúsculas**

- **Clases** en PascalCase: `DeliveryService`, no `deliveryService`.
- **Métodos y variables** en camelCase: `getDeliveryDescription`, no `DeliveryDescription`.
- **Constantes** en MAYUS_CON_GUION: `URGENT_DELIVERY_MULTIPLIER`.

NOMBRES **Errores de escritura**

Escribiste `Custumer` y `Costumer`. La correcta es `Customer`.

Copia el nombre de la clase que ya existe. Para renombrar en todo el proyecto usa **Shift + F6** en IntelliJ.

NOMBRES **Que digan la verdad**

- Todo el código en **inglés**: `precioCompraOriginal` → `originalPurchasePrice`.
- **Status ≠ Type**: el estado es `SaleStatus` y la modalidad es `DeliveryType`.
- 1.12 es un **multiplier**, no un `FEE_ADDED` (el recargo sería 0.12).
- El orden es **lo que la diferencia + el tipo**: `StandardDeliveryStrategy`, como `ArrayList`.
- Con varias implementaciones no se usa `...Impl`, porque no se distinguirían.
- Para los atributos, el nombre es el tipo en minúscula: `CustomerService customerService`.

COMPILA **La llamada y la firma deben coincidir**

Tienen que tener el mismo **nombre exacto**, la **cantidad** de parámetros y sus **tipos**.

```
// llamada
service.confirmSale(sale, delivery);
// firma
public SaleReceipt confirmSale(Sale sale,
DeliveryType delivery)
```

Te pasó: escribiste `DeliveryStrategy delivery` cuando se mandaba un `DeliveryType`.

COMPILA **Una variable que no existe**

Usaste `delivery` sin declararlo. Hay que declararlo **antes** y **dentro del mismo método**:

```
DeliveryType delivery = view.readDeliveryType();
```

COMPILA **La constante dentro del método**

`private static final` va **a nivel de clase**, arriba y fuera de los métodos.

Dentro de un método no se puede poner `private` ni `static`.

LÓGICA **Resultado perdido (BigDecimal es INMUTABLE)**

```
deliveryService.calculateFinalAmount(sale,
delivery); // se pierde
```

```
sale.setTotalAmount(deliveryService.calculateFinalA
mount(sale, delivery));
```

Pasa lo mismo con `setScale`, `add` y `multiply`: todos **devuelven** un valor nuevo que tienes que guardar.

BIGDECIMAL **Cómo crearlo**

- `new BigDecimal("1.12")` o `BigDecimal.valueOf(1.12)` → **correcto.**
- `new BigDecimal(1.12)` (con double, sin comillas) → valor impreciso.
- `BigDecimal x = 1.12;` → no compila (double ≠ BigDecimal).
- No hagas conversiones dobles: `new BigDecimal(String.valueOf(X))` cuando X ya es BigDecimal.

CAPAS **Cada cosa en su lugar**

- **Vista**: pregunta e imprime. No decide reglas de negocio.
- **Controlador**: coordina (lee de la vista → llama al servicio → muestra).
- **Servicio**: lógica y decisiones (caché o BD, cálculos, validaciones).
- **DAO**: solo SQL.
- En la estrategia o el servicio **no va println** de cara al usuario. La información sube hasta la vista.

SINTAXIS **Detalles que fallaste**

- `super` solo sirve con herencia (`extends`). Un método de la misma clase se llama por su nombre: `getStrategy(x)`.
- `return switch (...) { ... };` → lleva **;** después de la llave.
- El apóstrofo es `'`, no la tilde `´`.
- Deja un espacio antes de `{`: `implements X {`.

MÉTODO **Cuando te bloquees**

1. Escribe la receta en comentarios (paso 1, 2, 3...).
2. Busca en el proyecto algo parecido y copia el patrón (`DatabaseConnection`, `readCustomerType`...).
3. Arma las líneas difíciles de adentro hacia afuera.
4. Prueba con un `main` de prueba antes de integrarlo al menú.

2 · Plantillas rápidas (copiar y adaptar)

Singleton (3 ingredientes)

```
public class CustomerCache {
  private static CustomerCache instance;      // 1
  private CustomerCache() { }                 // 2
  public static CustomerCache getInstance() { // 3
    if (instance == null) instance = new
CustomerCache();
    return instance;
  }
}
// uso: CustomerCache.getInstance()   (nunca new)
```

Strategy

```
public interface DeliveryStrategy {
  BigDecimal calculateTotal(BigDecimal original);
}
public class UrgentDeliveryStrategy implements
DeliveryStrategy {
  private static final BigDecimal MULTIPLIER = new
BigDecimal("1.12");
  @Override
  public BigDecimal calculateTotal(BigDecimal original)
{
    return original.multiply(MULTIPLIER);
  }
}
// servicio: el switch SOLO elige
private DeliveryStrategy getStrategy(DeliveryType t) {
  return switch (t) {
    case STANDARD -> new StandardDeliveryStrategy();
    case URGENT   -> new UrgentDeliveryStrategy();
  };
}
```

Optional (la "caja")

Crear	Usar

`Optional.of(x)`: x no puede ser null
`Optional.empty()`: caja vacía
`Optional.ofNullable(x)`: decide sola
	`isPresent()` / `isEmpty()`
	`get()`: solo después de revisar
	`ifPresent(cache::put)`
	`map(Customer::getName).orElse("not found")`

`cache::put` es una referencia a método, la forma corta de `c -> cache.put(c)`.

BigDecimal y dinero

```
total.setScale(2, RoundingMode.HALF_UP)  // 373.3296 ->
373.33
a.add(b)  a.subtract(b)  a.multiply(b)    // no + - *
a.compareTo(b) == 0       // comparar valores (no
equals)
x.toPlainString()         // sin notación científica
x.stripTrailingZeros().toPlainString()   // 70.00 ->
"70"
```

Al multiplicar, los decimales se **suman**: 100000.00 × 1.12 = 112000.0000.

Map / Set / List

List	una fila, por posición, admite repetidos
Set	sin repetidos: `set.add(x)` devuelve **false** si ya estaba
Map	llave → valor: `Map<Integer, Customer>`, `put` / `get` / `containsKey`

JDBC (patrón de los DAO)

```
Connection conn =
DatabaseConnection.getInstance().getConnection();
try (PreparedStatement ps = conn.prepareStatement(SQL))
{
  ps.setDate(1, java.sql.Date.valueOf(localDate));
  try (ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
      lista.add(new
Fila(rs.getString("alias_del_select"), ...));
    }
  }
} catch (SQLException e) { throw new
RuntimeException("...", e); }
```

En `rs.getX("...")` va el **nombre de la columna o alias del SELECT**, no el de la variable Java.

Transacción (todo o nada)

```
DatabaseConnection.getInstance().inTransaction(() ->
    lista.forEach(dao::save));   // si uno falla ->
rollback
```

Archivos

```
Path f = Paths.get("clientes_importar.txt"); // raíz
proyecto
Files.exists(f)
List<String> lines = Files.readAllLines(f,
StandardCharsets.UTF_8);
try (BufferedWriter w = Files.newBufferedWriter(f,
UTF_8)) {
  w.write("a;b;c"); w.newLine();
}
"a;b;;".split(";", -1)   // -1 conserva vacíos del
final
LocalDate.now()          // toString() -> 2026-11-09
```

Streams (sin for/while)

```
lista.stream()
  .filter(r -> r.status() != SaleStatus.RETURNED)
  .collect(Collectors.groupingBy(Fila::categoria,
     Collectors.reducing(BigDecimal.ZERO,
Fila::cantidad, BigDecimal::add)));
// con int:  Collectors.summingInt(Fila::cantidad)
// contar:   Collectors.counting()
mapa.entrySet().stream().max(Map.Entry.comparingByValue
())  // -> Optional
lista.stream().map(Fila::cliente).distinct().sorted().t
oList();
```

Los getters de un **record** no llevan "get": se escribe `r.cantidad()`.

3 · Los 5 ejercicios: dónde está cada cosa y qué decir

Antes de probar, ejecuta `docs/practica_examen.sql` en Workbench (agrega el estado Devuelta y trae consultas de ayuda). En el código, todas las partes nuevas están marcadas con el comentario **EJERCICIO N**.

Ejercicio 1 · Caché de clientes (Singleton) · Ventas → opción 6

CustomerCache · CustomerService · SaleService.findCustomerById · CacheTest (main de prueba)

- **¿Por qué Singleton?** *"Para que SaleService y CustomerService compartan una sola caché. Aunque haya dos CustomerService, ambos usan getInstance()."*
- **¿Quién decide si se usa memoria o BD?** CustomerService, no la caché ni la vista. La caché es la nevera y solo guarda y devuelve.
- **Receta:** caché → si está, se devuelve (`[CACHE]`) → si no, BD (`[SQL]`) → si existe, `put` → se devuelve el Optional.
- **Casos de borde:** `put` rechaza null e id ≤ 0, y `ifPresent` no guarda el ID 999.
- **Prueba:** ejecuta `CacheTest`. Debe salir SQL, CACHE, SQL, CACHE, SQL + not found.

Ejercicio 2 · Reporte CSV de compras · Inventario → opción 10

DailyPurchase · PurchaseReportDao(Impl) · PurchaseReportService

- **Equivalencia (dila):** *"No hay tabla de compras: una compra es la recepción de un lote. El número es numero_lote, el proveedor sale del JOIN y el monto es cantidad_inicial × precio_compra."*
- **Fecha:** `LocalDate.now()`. En el SQL se usa `DATE(fecha_ingreso) = ?`, que quita la hora.
- **Monto:** `setScale(2, HALF_UP).toPlainString()`. Siempre lleva punto decimal, aunque Windows esté en español.
- Si no hay compras, se crea el archivo solo con la cabecera y se muestra un mensaje. Para tener compras de hoy usa Inventario → 3.

Ejercicio 3 · Tarifas con Strategy · Ventas → 3 (nueva venta)

DeliveryStrategy · Standard/UrgentDeliveryStrategy · DeliveryType · DeliveryService · StrategyTest

- **¿Interfaz o if?** *"Agregar una modalidad es crear una clase, sin tocar las que funcionan (abierto/cerrado)."*
- **¿Por qué no hay default?** Porque el switch sobre el enum cubre todos los casos. Si falta uno, no compila.
- **¿Por qué el redondeo va en el servicio?** Es una regla del dinero. Así, aunque una estrategia nueva olvide redondear, el resultado sale bien.
- La modalidad se pregunta **antes** de confirmar, y el total se ajusta **después** de `calculateTotals`.

Ejercicio 4 · Importar clientes · Ventas → opción 7

CustomerImportService · ImportException · clientes_importar.txt (raíz)

- **Todo o nada:** FASE 1 valida todas las filas y junta los errores. FASE 2 guarda todo en `inTransaction`: si una inserción falla, rollback.
- **Repetidos:** un `Set` detecta los repetidos en el archivo y `findByDocument` los que ya están en la BD.
- **Trampa:** usa `split(";", -1)`. Sin el -1, `"a;b;c;d;"` da 4 campos.
- Mapeo de tipos: MAYORISTA → WHOLESALE y MINORISTA → RETAIL. La línea vacía se salta con `isBlank()`.
- **Probar:** renombra el archivo (inexistente), cambia la cabecera, borra un campo y repite un documento. Si importas dos veces, la segunda falla por documento duplicado.

Ejercicio 5 · Estadísticas con Streams · Ventas → opción 8

SaleStatRow · SalesStatsDao(Impl) · CommercialStats · CommercialStatsService · SaleStatus.RETURNED(6)

- **JOIN:** ventas → detalles_venta → **lotes_inventario** → productos → categorias_producto, y ventas → clientes. Ojo: detalles_venta no tiene id_producto.
- **Equivalencia (dila):** *"CONFIRMADA = cualquier venta guardada que no esté cancelada ni devuelta. DEVUELTA = estado nuevo con id 6."*
- Para la categoría top se usa `filter → groupingBy + reducing → max(comparingByValue)`. Para los clientes, `filter → map → distinct`.
- Sin ventas, Optional vacío y mensaje informativo. Sin devoluciones, el apartado sale sin nombres.
- **Probar:** en el .sql, haz `UPDATE ventas SET id_estado = 6 WHERE id_venta IN (...)`.

4 · Si el examen cambia el escenario (lo más probable)

La guía dice que los exámenes son **equivalentes pero diferentes**. Lo que cambia es el dato; el patrón se queda igual:

Si piden...	Usa el mismo patrón de...	Cambia solo...
Caché de **productos** o **proveedores**	Ej. 1: `XCache` Singleton + `XService` que decide entre caché y BD	El tipo del Map: `Map<Integer, Product>` y su DAO
CSV de **ventas**, **mermas** o **facturas** del día	Ej. 2: DAO por fecha + servicio que escribe el archivo	El SQL (tabla y fecha), la cabecera y el nombre del archivo
Descuentos por tipo de cliente, **impuestos** o **métodos de pago**	Ej. 3: interfaz + una clase por caso + switch que solo elige	El enum, las clases y el porcentaje
Importar **productos** o **proveedores** desde un .txt o .csv	Ej. 4: validar todo, juntar errores y luego `inTransaction`	La cabecera, la cantidad de campos y las validaciones (NIT único, precio > 0...)
Estadísticas por **proveedor**, **producto** o **mes**	Ej. 5: JOIN → record → streams	La llave de `groupingBy` y el filtro

ORDEN **Cómo construir cualquier ejercicio**

1. Lee los **casos de prueba** primero: te dicen qué debe pasar.
2. Si hace falta, escribe el **SQL** en Workbench y pruébalo ahí.
3. Record o entidad → DAO (interfaz + Impl) → Servicio → main de prueba.
4. Al final, la vista (leer y mostrar) y el controlador (un `case` nuevo).
5. Si agregas una opción al menú, actualiza el número de **Back** y el `while (option != N)`.

Mensajes y errores

- Copia los mensajes del enunciado **al pie de la letra**, con tildes y punto final.
- Los errores esperados se lanzan como excepción en el servicio y se atrapan en el controlador. Así la app no se cierra y vuelve al menú.
- Los controladores ya atrapan `RuntimeException`; aprovéchalo.

Checklist antes de entregar

- ☐ Compila: sin rojo en IntelliJ y Build → Rebuild sin errores.
- ☐ La opción aparece en el menú y "Back" funciona.
- ☐ Probé **todos** los casos de prueba del enunciado.
- ☐ La app no se cae con datos malos (letras, archivo inexistente, ID que no existe).
- ☐ Revisé los nombres: PascalCase, camelCase, inglés y Customer bien escrito.
- ☐ Ningún cálculo quedó en la vista y ningún SQL quedó en el servicio.
- ☐ Sé explicar las 3 decisiones técnicas principales.

Frases para defender decisiones

- *"Usé BigDecimal porque double tiene errores de precisión con dinero."*
- *"Usé un enum y no un String para que un error de escritura no compile."*
- *"Usé Optional para obligar a manejar el caso 'no existe' y evitar NullPointerException."*
- *"La lógica va en el servicio y la vista solo muestra: así respeto las capas MVC."*
- *"Usé try-with-resources para que la conexión y el archivo se cierren solos."*
- *"En un proyecto real el [CACHE]/[SQL] sería un logger; aquí uso println para la demostración."*